



PUCE
Pontificia Universidad
Católica del Ecuador

PUCE VIRTUAL

Facultad de Ingeniería
Ingeniería en Ciberseguridad

DESARROLLO DE PLATAFORMAS

T1S4 · Unidad 4

Justificación técnica

**SecureGuard: API REST inicial y front-end
consumidor para la gestión de incidentes**

Express · Fetch · CRUD · middleware y contrato HTTP

Autor

Alejandro David Carriel Zambrano

5.º semestre · Back-end moderno, APIs REST y front-end consumidor

Quito — Ecuador · 2026

1. Para qué es esta entrega

Esta entrega junta lo de las Semanas 1 y 2 (el cliente) con el servidor HTTP nativo de la Semana 3 y lo pasa a una API REST con Express más un front que consume el contrato con Fetch. El caso de uso sigue siendo SecureGuard: listar incidentes, filtrarlos por severidad y registrar uno nuevo, sin que toda la lógica se quede viviendo solo en el navegador.

La rúbrica pedía rutas CRUD, controladores, modelo en memoria, middleware de registro, manejo de errores e interfaz integrada. Acá justifico las decisiones que hacen visible ese acuerdo entre cliente y servidor, o sea: qué ruta hace qué, qué código HTTP devuelve y por qué no dejé todo amontonado en un solo archivo.

2. De HTTP nativo a Express

En Semana 3 el servidor usaba `http.createServer` y yo comparaba método y pathname a mano. Sirvió para entender cómo entra una petición, pero mezclaba transporte, enrutado y respuesta en un solo archivo. Con Express esas piezas se separan: `express.json()` lee el cuerpo, `express.static` sirve `public/`, el router declara `/api/incidentes` y los controladores devuelven JSON.

El cliente deja de apuntar a un archivo y habla con un contrato HTTP. Esa es la diferencia entre un prototipo colgado en un puerto y una plataforma cuya presentación puede cambiar sin tener que reescribir el servidor cada vez que muevo un botón.

3. Capas del proyecto

`server.js` solo arranca y registra middlewares. Las rutas declaran GET, POST, PATCH y DELETE. El controlador traduce la petición a un caso de uso y elige el código HTTP. El servicio valida y orquesta. El modelo guarda un arreglo en memoria. El dominio construye el incidente (id, estado inicial, tipo).

El front conserva el MVC de Semana 2, pero `modelo.js` ya no lee un JSON local: llama a la API. Reutilicé dominio y validación; lo que cambió de verdad fue el transporte y el alcance del CRUD. No reescribí todo desde cero porque no hacía falta: la idea era mover la lógica al servidor, no inventar otra app.

4. Modelo en memoria y contrato REST

Al iniciar, el arreglo carga tres casos semilla. POST inserta al inicio; PATCH actualiza; DELETE no borra el rastro, cambia el estado a «Cerrado». En un SOC académico la baja lógica me parece más honesta que eliminar evidencia. La limitación la dejo clara: si se reinicia Node, se pierde lo creado después de la semilla. Eso es el alcance de la unidad, no un defecto que esté escondiendo.

Las rutas siguen el recurso: el listado acepta `?severidad=`; el detalle responde 404 si no existe; el alta responde 201; `/api/salud` verifica que el proceso está vivo.

Método	Ruta	Código	Responsabilidad
GET	<code>/api/salud</code>	200	Estado del servicio
GET	<code>/api/incidentes</code>	200	Listado; filtro opcional
GET	<code>/api/incidentes/:id</code>	200 / 404	Detalle
POST	<code>/api/incidentes</code>	201 / 400	Alta validada

Método	Ruta	Código	Responsabilidad
PATCH	/api/incidentes/:id	200 / 404	Actualización
DELETE	/api/incidentes/:id	200 / 404	Cierre lógico

Tabla 1. Contrato REST del recurso incidentes.

5. Middleware, errores y doble validación

El logger escribe método, URL y hora ISO. Con eso se ve que el filtro dispara GET /api/incidentes?severidad=alta y que el formulario hace POST. El manejador de errores evita que un throw deje al cliente sin JSON. Las rutas /api que no existen responden 404 con el mismo cuerpo { ok, mensaje, detalles }.

Los códigos no son adorno: 200 para lectura o cierre, 201 para alta, 400 para validación, 404 si no está, 500 si se cae algo interno. El servidor no confía en el navegador: el formulario valida para que la experiencia no sea horrible, y el servicio vuelve a validar título, descripción, severidad, tipo, fecha y correo. Si alguien manda basura con Postman, igual se corta.

6. Front-end consumidor y cierre

obtenerIncidentes arma la query del filtro, crearIncidente envía JSON e iniciar distingue carga, dato y error. El detalle reutiliza GET por id. Así se cumple listar, filtrar y crear con Fetch, sin tirar el HTML semántico de las primeras semanas.

Reutilicé dominio, validación e interfaz; sustituí el HTTP nativo, el JSON en disco y el CRUD a medias. Quedan fuera autenticación y base persistente, y está bien: no era el alcance de esta unidad. El valor de la entrega no es «ya tengo Node», sino un contrato que se puede demostrar con capturas de la UI, del JSON y de la bitácora.

SecureGuard en Semana 4 es la misma plataforma de las semanas anteriores, nada más que ahora tiene capas que sí se pueden señalar en una revisión: ruta, controlador, servicio, modelo y front que consume. Eso es lo que quería dejar visible.